THE UNIVERSITY of EDINBURGH

Bioinformatics Programming and System Management (PGBI11095)

ICA 2: Optional Python ICA

Exam Number: B300238

Table of Contents

User Manual 3
- **GitHub Links** 3
- **Introduction** 3
- **Installation and Setup** 3
- **Running the Programme (with Test Set)** 5

Maintenance Manual 9
- **Architecture Overview** 9
- **Module Descriptions** 9

User Manual

GitHub Links

GitHub Repository Link: https://github.com/B300238-2025/OptionalPythonICA.git

B300238-2025.OptionalPythonICA.tar.gz.cpt Encryption Key: **P1edPyper**

Introduction

This programme performs an automated protein sequence analysis pipeline, pulling data directly from the NCBI Protein database. Given a protein family and a taxonomic group, the pipeline retrieves protein sequences, performs multiple sequence alignment, conservation analysis, PROSITE motif scanning, and BLAST similarity searches, and saves the outputs into structured output directories.

The pipeline performs:

- NCBI sequence retrieval using E-utilities esearch and efetch,
- Multiple sequence alignment with EMBOSS Clustal Omega (ClustalO),
- Conservation analysis via EMBOSS Plotcon and consensus similarity calculation,
- PROSITE motif scanning using EMBOSS patmatmotifs,
- Local BLASTP analysis against a custom database, and
- Visualisation of conservation plots and BLAST hit landscape.

The user will be prompted for:

- An email (required for NCBI Entrez E-utilities usage),
- A protein family (Pfam name or ID),
- A taxonomic group (scientific name or taxon ID),
- Number of sequences for conservation analysis,
- Number of sequences for motif scanning, and
- Confirmations (yes/no) if necessary.

Installation and Setup

Follow these steps to set up the programme

1. Login/Open a new Linux shell terminal. Firstly, the user will have to download the programme into his/her own workspace.
2. Inside the terminal, type the following command lines, in order:
    a. *git clone https://github.com/B300238-2025/OptionalPythonICA.git* ➜ This will download the entire programme (a folder called "OptionalPythonICA") to the user's own workspace.
    b. *cd OptionalPythonICA* ➜ Changes the current working directory to the "OptionalPythonICA" directory.
3. Secondly, the "OptionalPythonICA" directory only contains one encrypted and zipped tar file called "B300238-2025.OptionalPythonICA.tar.gz.cpt". The user will have to decrypt, unzip, and extract all the contents from this file.
4. Inside the terminal, type the following command lines, in order:
    a. *ccrypt -d B300238-2025.OptionalPythonICA.tar.gz.cpt* ➜ The user will be prompted for

a decryption key. The decryption key is **P1edPyper**. When typing the decryption key, nothing will appear on the terminal. This is normal. The user has to ensure that the decryption key has been typed correctly before hitting the <Enter> key.

b. *gzip -d B300238-2025.OptionalPythonICA.tar.gz* ➜ Unzips the zipped tar file.

c. *tar xf B300238-2025.OptionalPythonICA.tar* ➜ Extracts all the contents from the tar file into the current working directory.

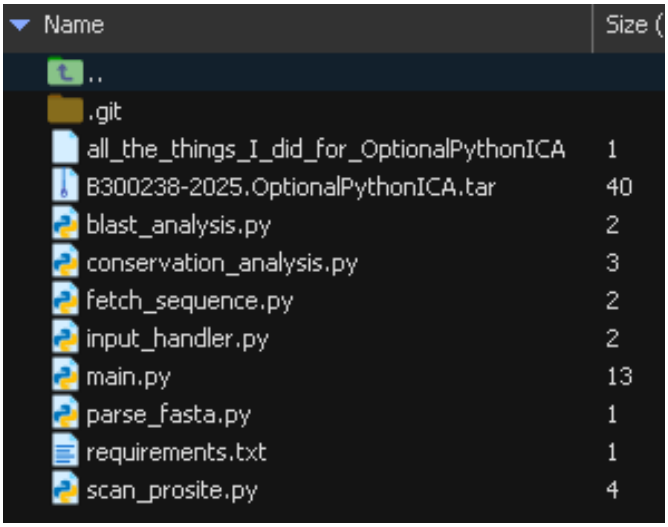

Figure 1: List of files after decrypting, unzipping, and extracting all contents from “B300238-2025.OptionalPythonICA.tar.gz.cpt”.

5. The extracted contents of the tar file will include: 7 Python (.py) files including main, input_handler, fetch_sequence, parse_fasta, conservation_analysis, scan_prosite, and blast_analysis, and a “requirements.txt” file (Figure 1). Additionally, there is a text file called “all_the_things_I_did_for_OptionalPythonICA” documenting the history of the creation of the programme. Lastly, the user will have to download the required dependencies of the programme, as documented by the “requirements.txt” file.
6. Inside the terminal, type the following command lines, in order:
   a. *python3 -m venv .venv* ➜ Creates a virtual environment locally to the current working directory. This is where the user will download the required dependencies for the programme.
   b. *source .venv/bin/activate* ➜ Activates the virtual environment. This should appear in the terminal: (.venv) username@machine:~/…(Figure 2)
   c. *pip install -r requirements.txt* ➜ Installs all the required dependencies of the programme, as documented by “requirements.txt” file (Figure 2).
7. Upon completing the above steps, the programme should be completely set up and ready to run.

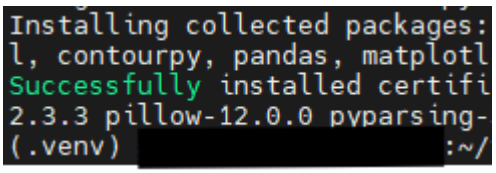

Figure 2: Terminal display after activating virtual environment and installing required dependencies of the programme.

Running the Programme (with Test Set)

The following is a demonstration of how to run the programme, using the test set: Glucose-6-phosphatase (a protein family) and Aves (a taxonomic group).

1. Before running the programme, the user has to ensure that the current working directory is set to the “OptionalPythonICA” directory containing all the Python (.py) files (Figure 1) and the virtual environment (.venv), and ensure that the virtual environment is activated (Figure 2).
2. To run the programme, type the following into the terminal: *./main.py*
3. The user will be prompted for the following: An email (required for NCBI Entrez E-utilities usage), a protein family (Pfam name or ID), and a taxonomic group (scientific name or taxon ID) (Figure 3). Entering any invalid input will cause the programme to reprompt the user.

```
(.venv)                    :~/test_OptionalICA/OptionalPythonICA$ ./main.py
Your email (needed for NCBI URL request): no

Please input a valid email.
Your email (needed for NCBI URL request):
Protein family: I don't know

Please input a valid protein family. No result for I don't know on EBI Search!
Protein family: glucose-6-phosphatase
Taxon group: I don't know

Please input a valid taxon group. No result for I don't know on NCBI Taxonomy!
Taxon group: Aves

Success! Your protein family: PAP2      Pfam ID: PF01569
Success! Your taxon group: Aves Taxon ID: 8782

Searching NCBI Protein...
Your dataset contains more than 1000 protein sequences!
Are you sure you want to continue? (y/n)
```

Figure 3: Inputting email, protein family, and taxon group.

4. If the user’s dataset has been found to contain more than 1000 protein sequences, the programme will prompt the user for a confirmation to continue. If no, the programme will be discontinued. If yes, the programme will continue to process the dataset. This test set contains more than 1000 protein sequences.
5. The programme will attempt to retrieve and download all protein sequences associated with the user’s inputted protein family and taxonomic group. This may take a while depending on the size of the dataset.
6. Once all the protein sequences have been successfully downloaded and parsed, the programme will prompt the user for a sample size of protein sequences to be analysed with ClustalO. In this pipeline, the sample size is assumed to be the number of sequences with the largest lengths. It is recommended to input a size within a range of 500 to 1000. For this test set, the user inputs a sample size of 1500. If the input exceeds the recommended range, the user will be prompted for another confirmation (Figure 4)

```
Please input a number for the number of sequences to be analysed with ClustalO. Maximum: 4999
The top few sequences with the largest lengths will be used for conservation analysis.
Note: The larger the number, the slower the analysis. Recommended: 500-1000
Sample size: one thousand five hundred

Invalid input. invalid literal for int() with base 10: 'one thousand five hundred'
Please input a number for the number of sequences to be analysed with ClustalO. Maximum: 4999
The top few sequences with the largest lengths will be used for conservation analysis.
Note: The larger the number, the slower the analysis. Recommended: 500-1000
Sample size: 1500

Your input exceeds the recommended range. Are you sure you want to continue? (y/n) yessir

Invalid input.

Your input exceeds the recommended range. Are you sure you want to continue? (y/n) y

Your sample size: 1500
The 1500 longest protein sequences will be used for conservation analysis.

Performing conservation analysis with ClustalO and Plotcon...
```

Figure 4: Inputting sample size for ClustalO analysis.

7. The programme will automatically run conservation analysis with ClustalO, and plot and summarise data with Plotcon and other EMBOSS utilities. Similarly, this may take a while depending on the sample size.
8. A similarity plot of aligned sequences will be displayed on a separate window (Figure 5). Check the plot, and simply close the window to continue.

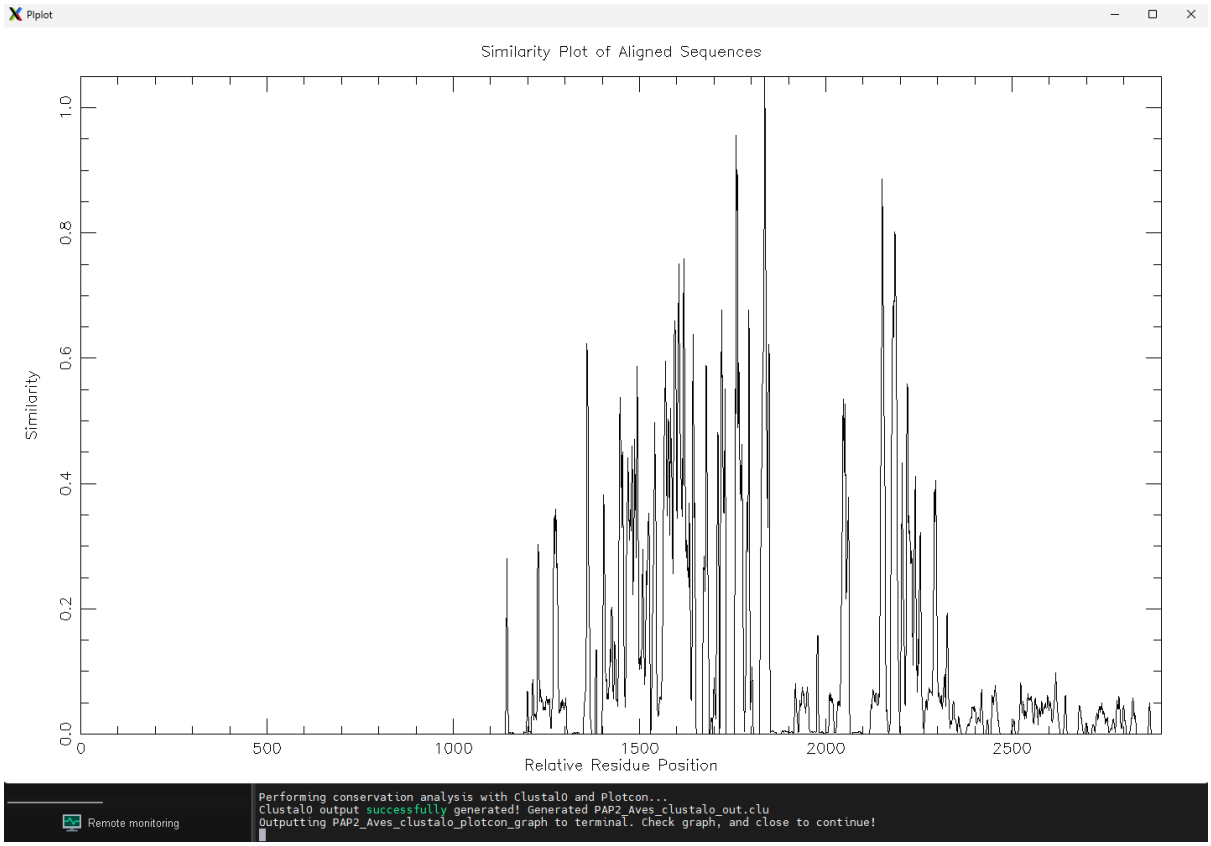

Figure 5: Similarity plot of alignment of the top 1500 longest sequences from protein family glucose-6-phosphatase and taxon group Aves.

9. Lastly, after finishing conservation analysis, the programme will prompt the user for a scan size for the number of protein sequences to be scanned for PROSITE motifs. In this pipeline, the

scan size is assumed to be the number of sequences with the highest similarities to the consensus sequence generated by ClustalO. For this test set, the user inputs a scan size of 200 (Figure 6).

```
Please input a number for the number of sequences to be scanned for PROSITE motifs. Maximum: 1500
The top few sequences with the highest similarity with the consensus sequence will be scanned for PROSITE motifs.
Scan size: 2000

Invalid input. Invalid input exceeds maximum size.
Please input a number for the number of sequences to be scanned for PROSITE motifs. Maximum: 1500
The top few sequences with the highest similarity with the consensus sequence will be scanned for PROSITE motifs.
Scan size: 200

Your scan size: 200
The 200 protein sequences with the highest similarity to the consensus sequence will be scanned for PROSITE motifs.

Scanning protein sequences for PROSITE motifs...
```

Figure 6: Inputting scan size for PROSITE motif scan.

10. A count plot of PROSITE motifs found will be displayed on a separate window (Figure 7). Check the plot, and simply close the window to continue.

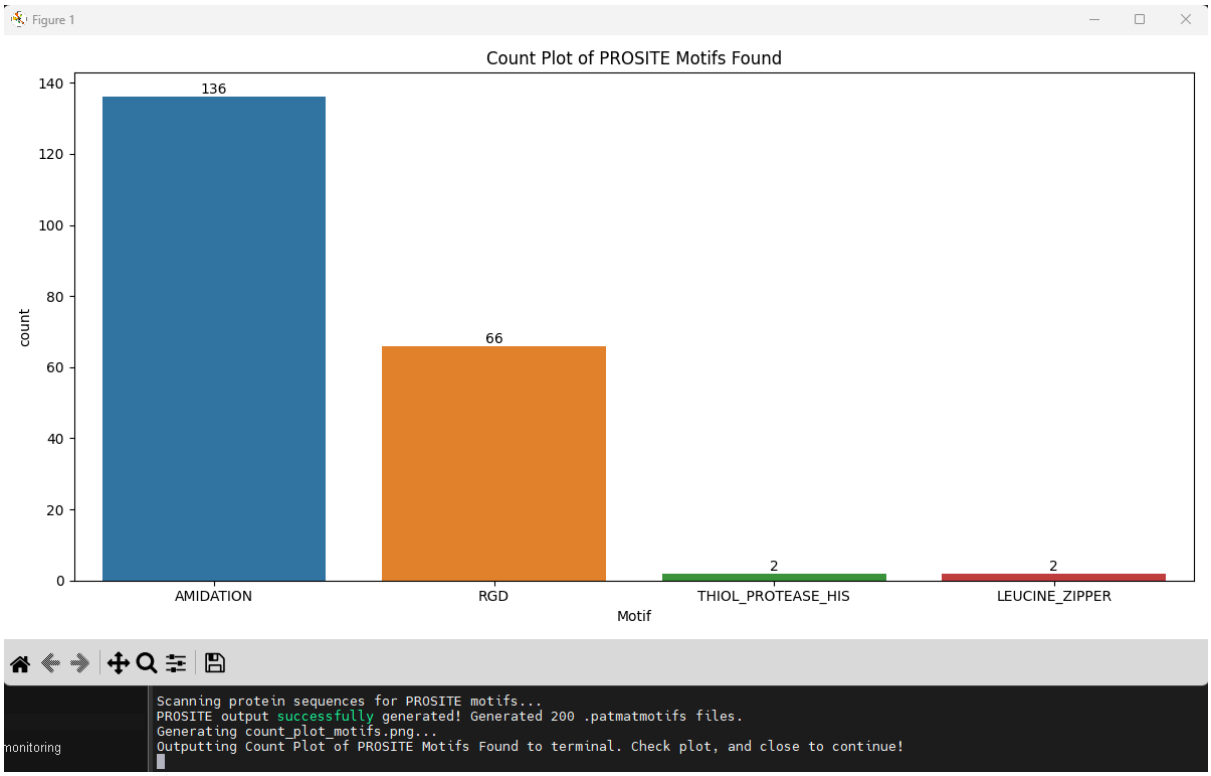

Figure 7: Count plot of the number of each PROSITE motif found in the top 200 sequences with the highest similarities to the consensus sequence.

11. Finally, after scanning selected sequences for PROSITE motifs, the programme will automatically run BLAST analysis of all the protein sequences against the reference sequence, which is the sequence with the highest similarity to the consensus sequence generated by ClustalO. A scatterplot of the BLAST hits landscape will be displayed on a separate window (Figure 8). Check the plot, and simply close the window to continue.

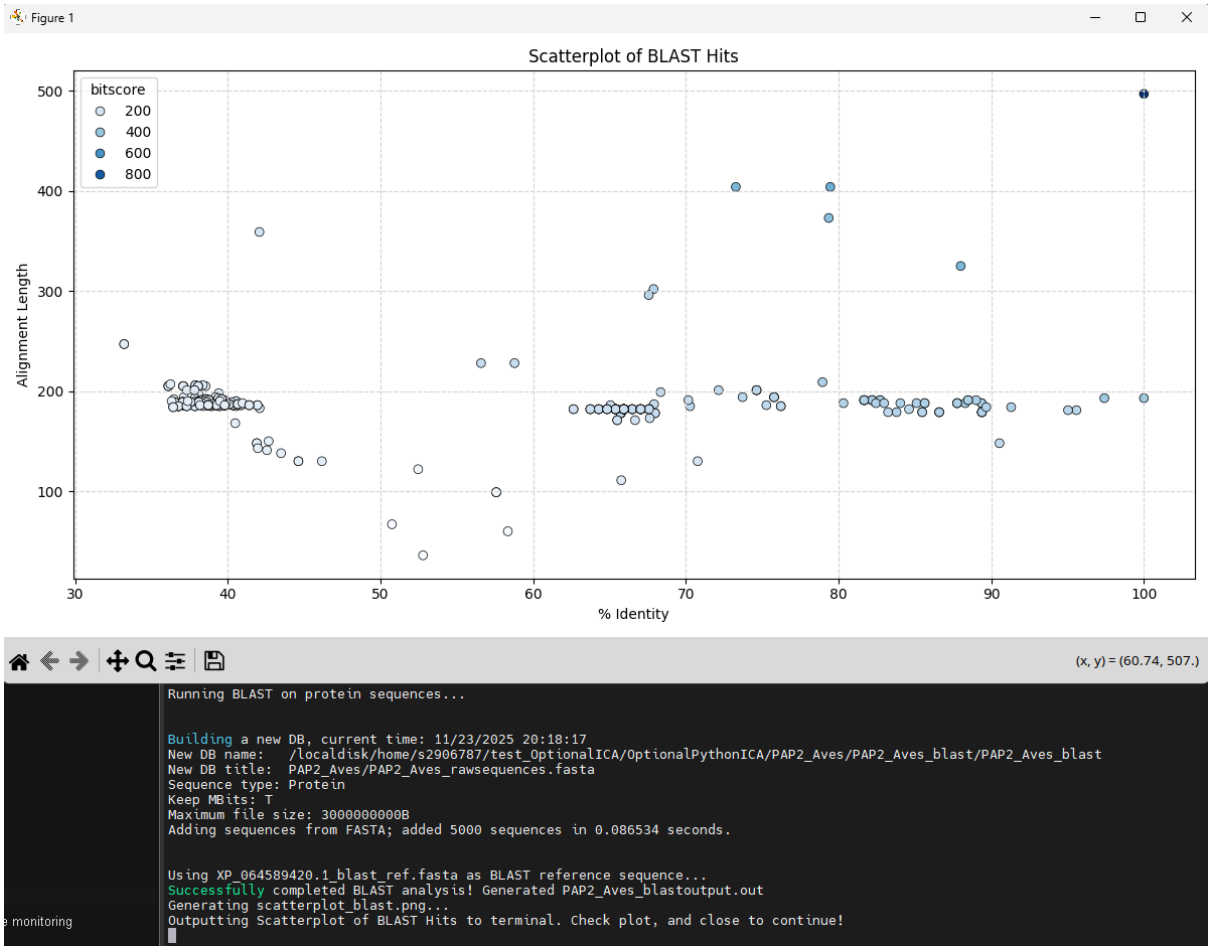

Figure 8: Scatterplot of BLAST hits landscape against the reference sequence with the highest similarity to the consensus sequence.

12. The pipeline is finished. All outputs are saved to a custom directory named “{pfam_name}_{taxon_name}”. For this test set, all outputs are saved to “PAP2_Aves” directory.

Maintenance Manual

Architecture Overview

The “main.py” script runs the pipeline, which performs a sequential execution of various modules: “input_handler.py”, “fetch_sequence.py”, “conservation_analysis.py”, “scan_prosite.py”, and “blast_analysis.py”, in this order (Figure 9). All outputs are saved to a custom directory named “{pfam_name}_{taxon_name}”, as inputted by the user.

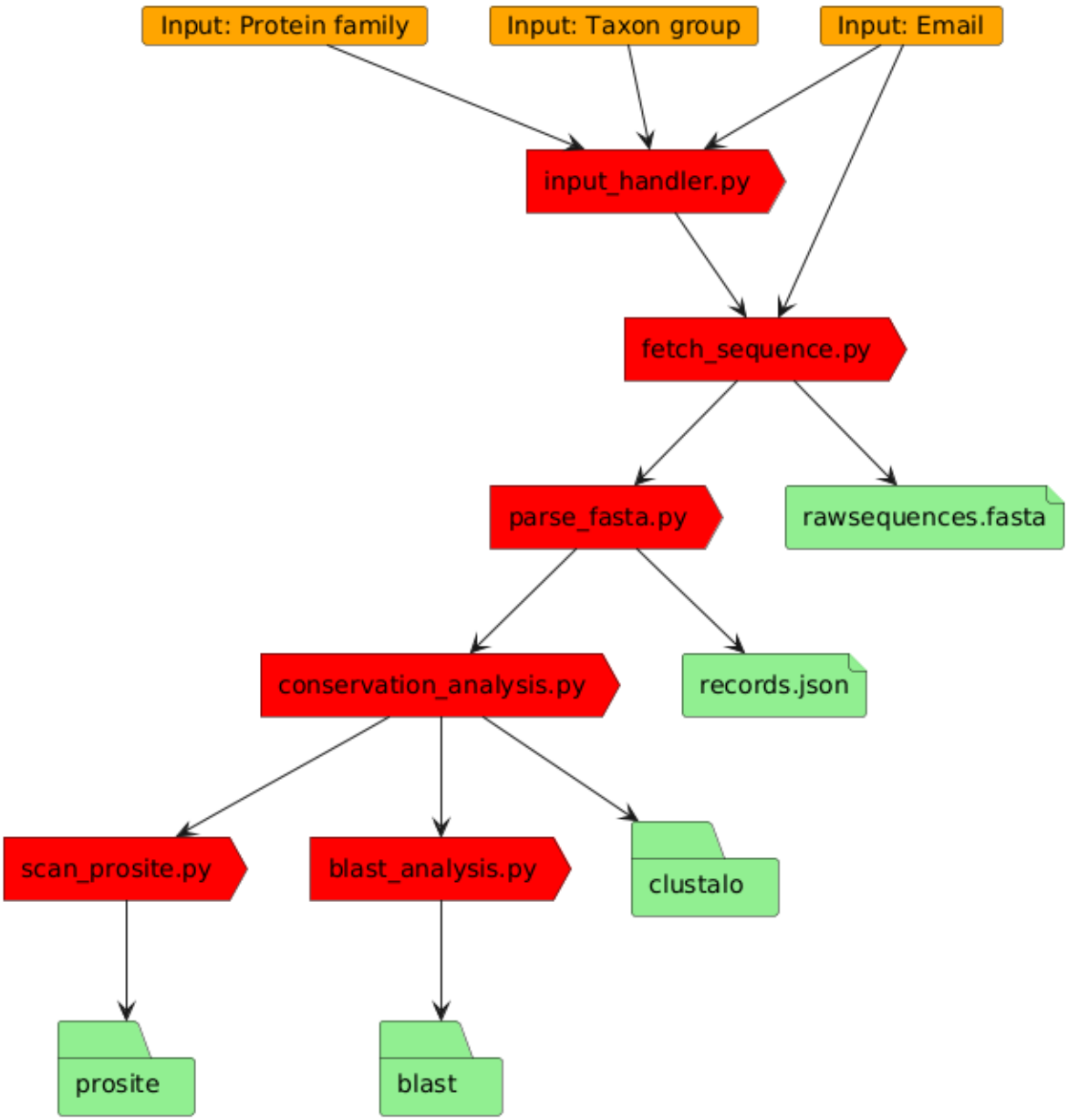

Figure 9: Summary flowchart of pipeline.

Module Descriptions

input_handler.py ➜ Validates the inputs: protein family and taxon group, by searching for their existence on Pfam and NCBI Taxonomy databases respectively

- *validate_protein(protein_family)* ➜ Queries EBI search with *protein_family* using REST API. If

query failure or no results, raise an error. Else, return the first Pfam ID and Pfam name.

- *validate_taxon(taxon_group, email)* ➜ Queries NCBI Taxonomy database with *taxon_group* using REST API. If query failure or no results, raise an error. Else, return the first taxon ID and taxon group name.

- Interaction with main.py: If an error is raised, reprompt user for another input. Else, return matched Pfam and Taxonomy IDs and names.

fetch_sequence.py ➜ Queries NCBI Protein database using Entrez esearch and efetch to get all protein sequences associated with user's inputted protein family and taxon group. Downloads all protein sequences into a single FASTA file.

- *run_esearch(protein_family, taxon_id, email)* ➜ Queries NCBI Protein database using Entrez esearch REST API to get a list of unique IDs. If query failure or no results, print error message and return an empty list. Else, return a list of unique IDs

- *run_efetch(ids_list, out_dir, email)* ➜ With the list of unique IDs obtained from esearch, query NCBI Protein database using Entrez efetch REST API to get a raw FASTA file of all protein sequences. If query failure, print error message and exit system. Else, return the downloaded raw FASTA filename

- Interaction with main.py: If query failure or no results, exit the system. Else, return the downloaded raw FASTA filename.

parse_fasta.py ➜ Parse the downloaded raw FASTA file with all sequences into JSON format, through *parse_fasta(out_dir, fasta_filename)*. Outputs a JSON file and all the protein sequences in JSON format.

- Interaction with main.py: Outputs a list of dictionaries of each protein sequence in JSON format

conservation_analysis.py ➜ Performs conservation analysis of a sample size of protein sequences inputted by user. This sample size refers to the top few longest protein sequences. Conservation analysis is performed using EMBOSS Clustal Omega (ClustalO) with multi-sequence alignment, visualised using EMBOSS Plotcon, and summarised using EMBOSS *cons* and *infoalign*.

- *build_clustalo_input(records, out_dir, out_clustalo, size)* ➜ Prepares an input FASTA file for ClustalO. As per user's input size, sort the sequences by length in descending order, and select the top few longest sequences for conservation analysis. The selected sequences are parsed into a single FASTA file.

- *run_clustalo(out_dir, out_clustalo, clustalo_input_filename)* ➜ Runs *clustalo* command line for multi-sequence alignment to output a Multi-Sequence Alignment(MSA) output file with .clu.

- *run_plotcon(out_dir, out_clustalo, clustalo_output_filename)* ➜ Runs *plotcon* command lines to generate Plotcon graph to terminal and save graph as .png file. Additionally, runs plotcon command line to generate Plotcon raw data and save as .txt file

- *get_consensus(out_dir, out_clustalo, clustalo_output_filename)* ➜ Runs *cons* command line to obtain the consensus sequence from MSA output.

- *get_infoalign(out_dir, out_clustalo, clustalo_output_filename)* ➜ Runs *infoalign* command line to obtain percentage similarity of each sequence to the consensus sequence generated by ClustalO.

- Interaction with main.py: Outputs the filenames of all output files.

scan_prosite.py ➜ Scans selected protein sequences for PROSITE motifs using EMBOSS *patmatmotifs*. The user's inputted scan size refers to the top few protein sequences with the highest percentage similarities to the consensus sequence generated by ClustalO.

- *parse_clu_results(records, out_dir, out_clustalo, infoalign_results, out_prosite, size)* ➜ Filters and selects the top few sequences with the highest percentage similarity to the consensus sequence from *infoalign_results*. Returns the top sequences in JSON and *infoalign_results*.

- *build_prosite_input(records_top, out_dir, out_prosite)* ➜ Prepares the individual FASTA files of each selected sequence for input into *patmatmotifs*. Returns a list of input FASTA files(filepaths) for *patmatmotifs*.

- *run_prosite_scan(out_dir, out_prosite, prosite_input_list)* ➜ Runs *patmatmotifs* command line for each input FASTA file, to scan each sequence for PROSITE motifs. Outputs a .patmatmotifs file for each corresponding .fasta file. Returns a list of output *patmatmotifs* files.

- *parse_prosite_output(out_dir, out_prosite, prosite_output_list)* ➜ Parses output *patmatmotifs* files into a summary table, and plots and saves a count plot of PROSITE motifs found. Additionally, returns a list of unique PROSITE motifs found.

- Interaction with main.py: Outputs the filenames of all output files, a pandas dataframe of the summary table, and a list of unique PROSITE motifs found.

blast_analysis.py ➜ Conducts BLAST analysis of all protein sequences against the reference sequence which is the sequence with the highest percentage similarity to the consensus sequence generated by ClustalO.

- *make_blast_db(out_dir, records, blast_db)* ➜ Constructs a FASTA file containing all protein sequences from records which is a list of dicts of sequences. Runs *makeblastdb* command line to construct a BLAST database of all the protein sequences.

- *select_blast_ref(top_records, top_df, out_dir, blast_db)* ➜ Selects the sequence with the highest percentage similarity with the consensus sequence and saves the sequence in a FASTA file.

- *run_blast_search(out_dir, blastref_filename, blast_db)* ➜ Runs *blastp* command line to conduct BLAST analysis of the custom BLAST protein database with the reference protein sequence query. Outputs the blastoutput.out file.

- *parse_blast_output(out_dir, blast_db, blastresults_filename)* ➜ Parses BLAST output file to a pandas dataframe, and visualise BLAST output by plotting a scatterplot of BLAST hits (Alignment Length against % Identity). Outputs a scatterplot of BLAST hits and parsed BLAST output to pandas dataframe.

- Interaction with main.py: Outputs the filenames of all output files, and a pandas dataframe of parsed BLAST output.